

Отчёт по задаче «Китайский почтальон»

Смола Артём, группа 23.Б08

1 Постановка задачи

Дан связный неориентированный взвешенный граф. Требуется найти минимальную длину замкнутого маршрута, который проходит по каждому ребру графа хотя бы один раз.

2 Принцип решения

Если все вершины имеют чётную степень, граф эйлеров, и ответ равен сумме длин всех рёбер. Если есть вершины нечётной степени, часть рёбер нужно пройти повторно.

Алгоритм:

1. Посчитать сумму длин всех рёбер.
2. Найти множество вершин нечётной степени.
3. Для всех нечётных вершин посчитать попарные кратчайшие расстояния.
4. Для поиска расстояний использовать алгоритм Дейкстры.
5. Найти минимальное попаривание нечётных вершин методом динамики по битовой маске.
6. Ответ:

$$\text{sum_edges} + \text{min_matching_cost}.$$

В данной реализации кратчайшие пути ищутся алгоритмом Дейкстры от каждой нечётной вершины. Это позволяет не считать полную матрицу всех пар вершин, как в алгоритме Флойда, и обычно даёт лучшую практическую скорость на разреженных графах.

3 Корректность

Если все степени вершин чётные, граф эйлеров. Тогда существует маршрут, проходящий по каждому ребру ровно один раз, а его длина равна сумме рёбер.

Если есть нечётные вершины, в любом эйлеровом дополнении они должны быть разбиты на пары добавленными проходами по путям. Минимальная добавка равна стоимости минимального попаривания нечётных вершин по метрике кратчайших путей.

Динамическое программирование по битовой маске перебирает все корректные попаривания и выбирает минимум. Значит, итог

$$\text{sum_edges} + \text{min_matching_cost}$$

оптимален.

4 Ограничения реализации

- число нечётных вершин ограничено: $k \leq 22$;
- при $k > 22$ программа завершает работу с ошибкой;
- веса рёбер предполагаются неотрицательными.

5 Исходный код

Исходный код программы и тестовые данные доступны в репозитории GitHub: https://github.com/artem-smola/formal_lang.

6 Код программы

Код программы:

```
#include <algorithm>
#include <iostream>
#include <limits>
#include <queue>
#include <utility>
#include <vector>

struct Edge {
    int to;
    long long w;
};

struct MatchInfo {
    int first = -1;
    int second = -1;
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int n = 0;
    int m = 0;
    if (!(std::cin >> n >> m)) {
        std::cerr << "Ошибка: не удалось прочитать n и m.\n";
        return 1;
    }
    if (n <= 0 || m < 0) {
        std::cerr << "Ошибка: некорректные n или m.\n";
        return 1;
    }

    std::vector<std::vector<Edge>> graph(n + 1);
    std::vector<int> degree(n + 1, 0);
    long long base_sum = 0;

    for (int i = 0; i < m; ++i) {
        int u = 0;
        int v = 0;
        long long w = 0;
```

```

    if (!(std::cin >> u >> v >> w)) {
        std::cerr << "Ошибка: не удалось прочитать ребро на строке " << (i +
            ↪ 1) << ".\n";
        return 1;
    }
    if (u < 1 || u > n || v < 1 || v > n || w < 0) {
        std::cerr << "Ошибка: некорректное ребро на строке " << (i + 1) <<
            ↪ ".\n";
        return 1;
    }

    graph[u].push_back({v, w});
    graph[v].push_back({u, w});
    degree[u]++;
    degree[v]++;
    base_sum += w;
}

// Проверка связности по вершинам с ненулевой степенью.
int start = -1;
for (int v = 1; v <= n; ++v) {
    if (degree[v] > 0) {
        start = v;
        break;
    }
}
if (start != -1) {
    std::vector<int> stack = {start};
    std::vector<char> used(n + 1, 0);
    used[start] = 1;

    while (!stack.empty()) {
        int v = stack.back();
        stack.pop_back();
        for (const auto& e : graph[v]) {
            if (!used[e.to]) {
                used[e.to] = 1;
                stack.push_back(e.to);
            }
        }
    }

    for (int v = 1; v <= n; ++v) {
        if (degree[v] > 0 && !used[v]) {
            std::cerr << "Ошибка: граф несвязный, задача не определена.\n";
            return 1;
        }
    }
}

std::vector<int> odd;
for (int v = 1; v <= n; ++v) {
    if (degree[v] % 2 == 1) {
        odd.push_back(v);
    }
}

if (odd.empty()) {
    std::cout << "Сумма длин всех ребер: " << base_sum << "\n";
    std::cout << "Добавочная длина: 0\n";
}

```

```

std::cout << "Минимальная длина маршрута почтальона: " << base_sum <<
    ↪ "\n";
std::cout << "Все степени четные, дополнительное дублирование не
    ↪ требуется.\n";
return 0;
}

const int k = static_cast<int>(odd.size());
if (k > 22) {
    std::cerr << "Ошибка: слишком много вершин нечетной степени (" << k
        << "), битмаска DP слишком большая для этой реализации.\n"
        << "Текущий лимит реализации: k <= 22.\n";
    return 1;
}

const long long INF = std::numeric_limits<long long>::max() / 4;
std::vector<std::vector<long long>> dist_odd(k, std::vector<long long>(k,
    ↪ INF));

// Кратчайшие пути от каждой нечетной вершины до всех вершин графа.
for (int i = 0; i < k; ++i) {
    const int src = odd[i];
    std::vector<long long> dist(n + 1, INF);
    dist[src] = 0;

    using State = std::pair<long long, int>;
    std::priority_queue<State, std::vector<State>, std::greater<State>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [cur_dist, v] = pq.top();
        pq.pop();
        if (cur_dist != dist[v]) {
            continue;
        }

        for (const auto& e : graph[v]) {
            if (dist[e.to] > cur_dist + e.w) {
                dist[e.to] = cur_dist + e.w;
                pq.push({dist[e.to], e.to});
            }
        }
    }

    for (int j = 0; j < k; ++j) {
        dist_odd[i][j] = dist[odd[j]];
        if (dist_odd[i][j] >= INF / 2) {
            std::cerr << "Ошибка: между нечетными вершинами нет пути.\n";
            return 1;
        }
    }
}

const int max_mask = 1 << k;
std::vector<long long> dp(max_mask, INF);
std::vector<MatchInfo> parent(max_mask);
dp[0] = 0;

for (int mask = 1; mask < max_mask; ++mask) {
    if (__builtin_popcount(static_cast<unsigned>(mask)) % 2 == 1) {

```

```

        continue;
    }

    int i = 0;
    while (i < k && ((mask & (1 << i)) == 0)) {
        ++i;
    }
    if (i == k) {
        continue;
    }

    const int mask_without_i = mask ^ (1 << i);
    for (int j = i + 1; j < k; ++j) {
        if ((mask_without_i & (1 << j)) == 0) {
            continue;
        }
        const int next_mask = mask_without_i ^ (1 << j);
        const long long candidate = dp[next_mask] + dist_odd[i][j];
        if (candidate < dp[mask]) {
            dp[mask] = candidate;
            parent[mask] = {i, j};
        }
    }
}

const int full_mask = max_mask - 1;
const long long extra = dp[full_mask];
if (extra >= INF / 2) {
    std::cerr << "Ошибка: не удалось найти корректное попаривание нечетных
↪ вершин.\n";
    return 1;
}

long long answer = base_sum + extra;
std::cout << "Сумма длин всех ребер: " << base_sum << "\n";
std::cout << "Добавочная длина: " << extra << "\n";
std::cout << "Минимальная длина маршрута почтальона: " << answer << "\n";

std::cout << "Попаривание нечетных вершин:\n";
int mask = full_mask;
while (mask != 0) {
    const int i = parent[mask].first;
    const int j = parent[mask].second;
    if (i < 0 || j < 0) {
        std::cerr << "Ошибка восстановления решения.\n";
        return 1;
    }
    std::cout << "  " << odd[i] << " - " << odd[j]
        << " (кратчайший путь = " << dist_odd[i][j] << ")\n";
    mask ^= (1 << i);
    mask ^= (1 << j);
}

return 0;
}

```

7 Демонстрация работы программы

1) Входные данные

Рассматриваемый тест: test2.

Формат входных данных:

1. n, m — число вершин и число рёбер.
2. Далее m строк вида $u v w$, где u и v — концы ребра, $w \geq 0$ — вес ребра.

Ограничения: граф неориентированный, а вершины с ненулевой степенью лежат в одной компоненте связности.

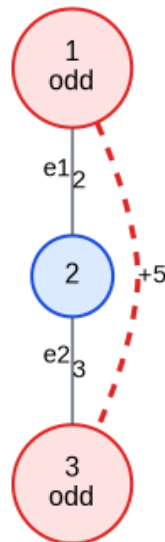
```
3 2
1 2 2
2 3 3
```

2) Вывод

```
artem@artem-ВМН-WCX9:~/ТП_6_семестр/ТФЯИТ/Задания/Chinese_Postman/Solution/Code$ ./postman < tests/test2.in
Сумма длин всех ребер: 5
Добавочная длина: 5
Минимальная длина маршрута почтальона: 10
Попаривание нечетных вершин:
 1 - 3 (кратчайший путь = 5)
```

3) Визуализация

Chinese Postman: test2; sum=5, extra=5, total=10; odd vertices: 1, 3



4) Описание результата

Для теста test2:

- сумма длин всех рёбер равна 5;
- добавочная длина равна 5;

- минимальная длина маршрута почтальона равна 10;
- для нечётных вершин выбрано попаривание $1 \leftrightarrow 3$.

Это соответствует классическому решению задачи китайского почтальона: к сумме рёбер добавляется минимальная стоимость попаривания нечётных вершин.

5) Дополнительный граничный тест

Для **test1** все степени вершин уже чётные, поэтому:

Входные данные теста **test1**:

```
4 4
1 2 3
2 3 4
3 4 5
4 1 6
```

добавочная длина = 0, ответ = сумма рёбер.

Это проверяет граничный случай эйлера графа, где дублирование рёбер не требуется.

8 Анализ сложности

Пусть:

- n — число вершин;
- m — число рёбер;
- k — число вершин нечётной степени (k всегда чётно).

Кратчайшие пути от каждой нечётной вершины:

$$O(k \cdot m \log n).$$

Динамика по маске для минимального попаривания:

$$O(2^k \cdot k^2).$$

Итоговая сложность:

$$O(k \cdot m \log n + 2^k \cdot k^2).$$

Память:

$$O(n + m + 2^k).$$